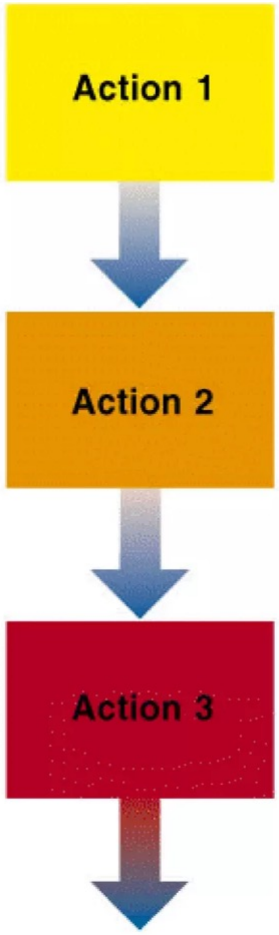
Control Structures

Control Structures

- Programs are written using three basic structures
  - Sequence
    - a sequence is a series of statements that execute one after another
  - Repetition(loop or iteration)
    - repetition (looping) is used to repeat statements while certain conditions are met.
  - Selection(branching)
    - selection (branch) is used to execute different statements depending on certain conditions
- Called control structures or logic structures

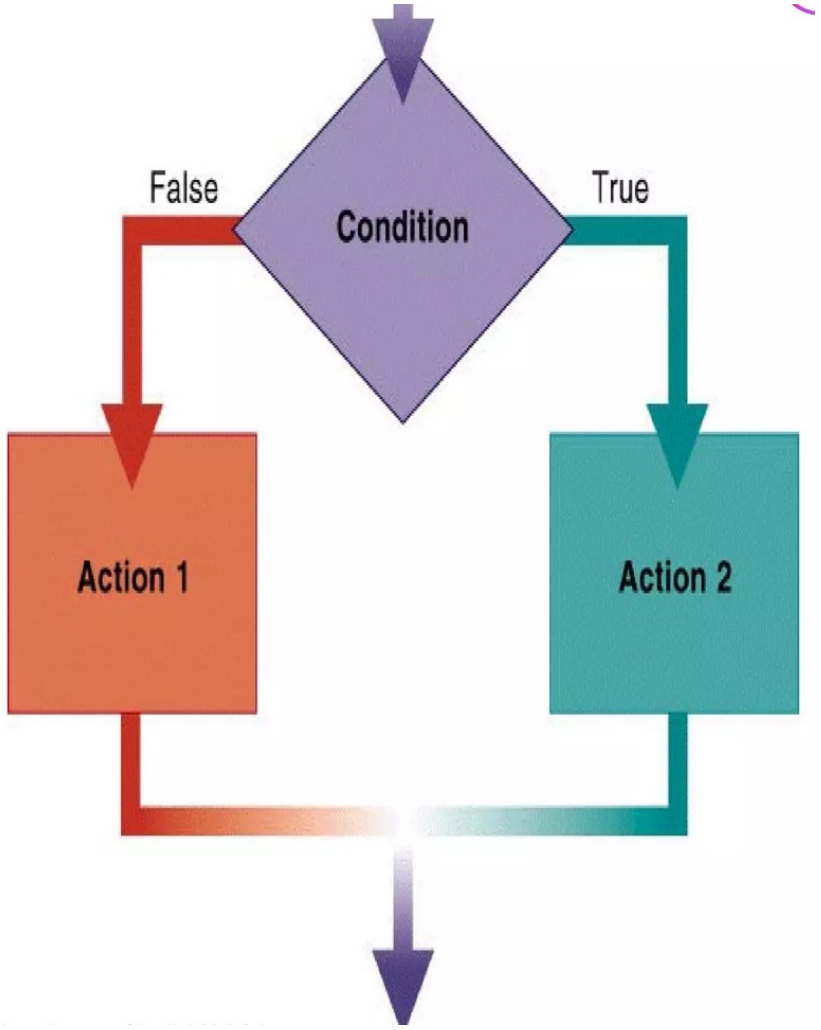
Sequence

- The **sequence structure** directs the computer to process the program instructions, one after another, in the order listed in the program



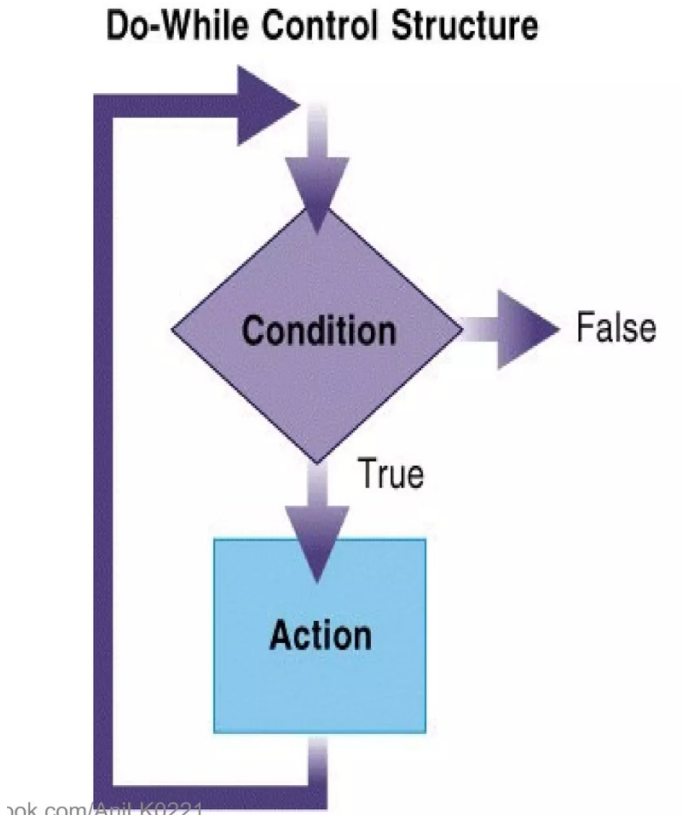
Selection (Branch)

- **Selection structure:** makes a decision and then takes an appropriate action based on that decision
  - Also called the **decision structure**



Repetition (loop)

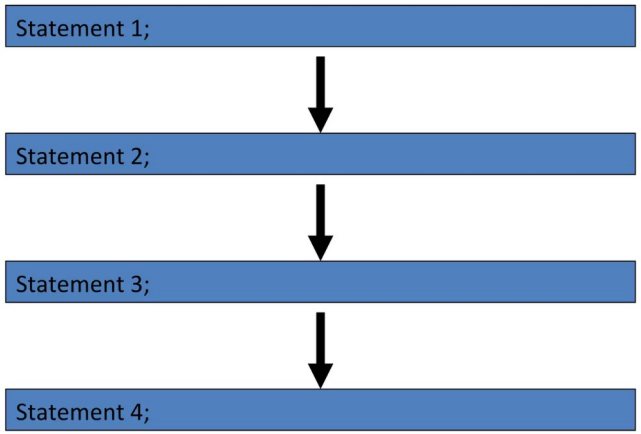
- **Repetition structure:** directs computer to repeat one or more instructions until some condition is met
  - Also called a **loop** or **iteration**



Flow Control

- How the computer moves through the program
- Many keywords are for "flow control"

Normal flow



Flow control

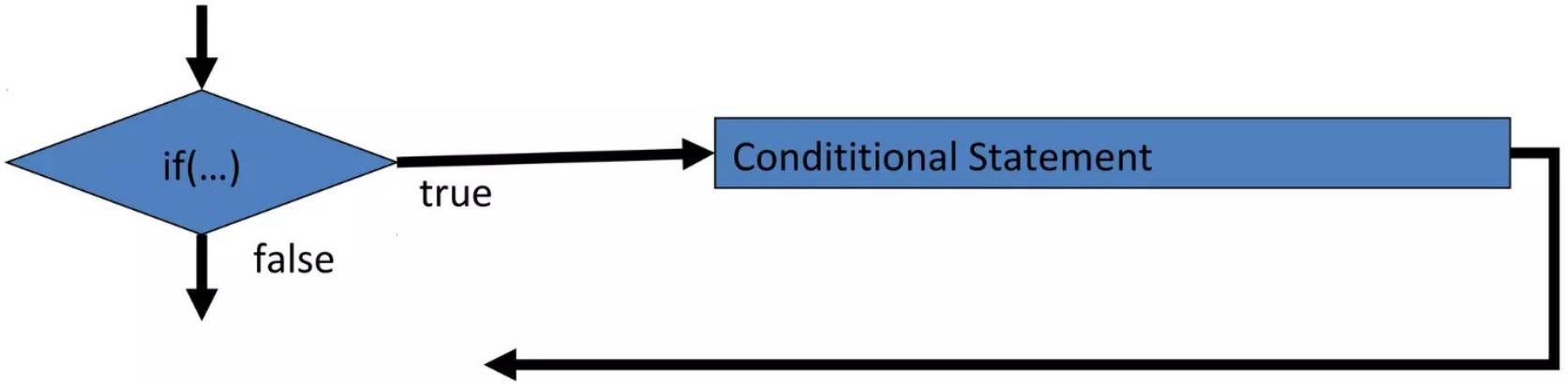
- Selection (Branch)
  - if
  - if-else
  - nested if
  - (goto)
  - switch
- Repetition (Loop)
  - while
  - do-while
  - For
  - nesting

Conditional Statements

- if
- if else
- nested if (if – else if – else if – else)
- statement blocks ({…})
- (goto)
- switch (case, default, break)

if

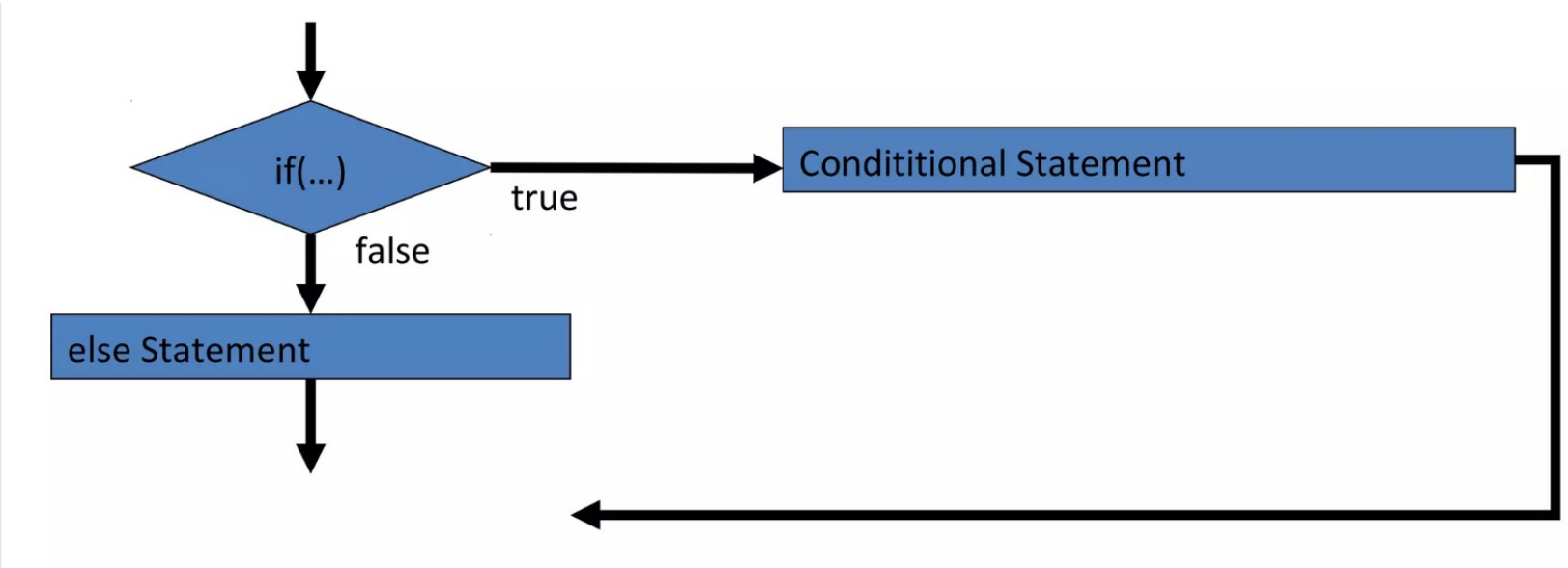
- if(condition)statement
  - If the condition is true, statement is executed.
  - If the condition is false, statement is not executed.



if else

- if (condition)statement1
  else statement2

- If the statement1 is true, then print out on the screen **x is 100**
- If the statement2 is true, then print out on the screen **x is not 100**



Statement Block

- If we want more than a single instruction is executed, we must group them in a in a block of statements by using curly brackets ({…})

Nested statements

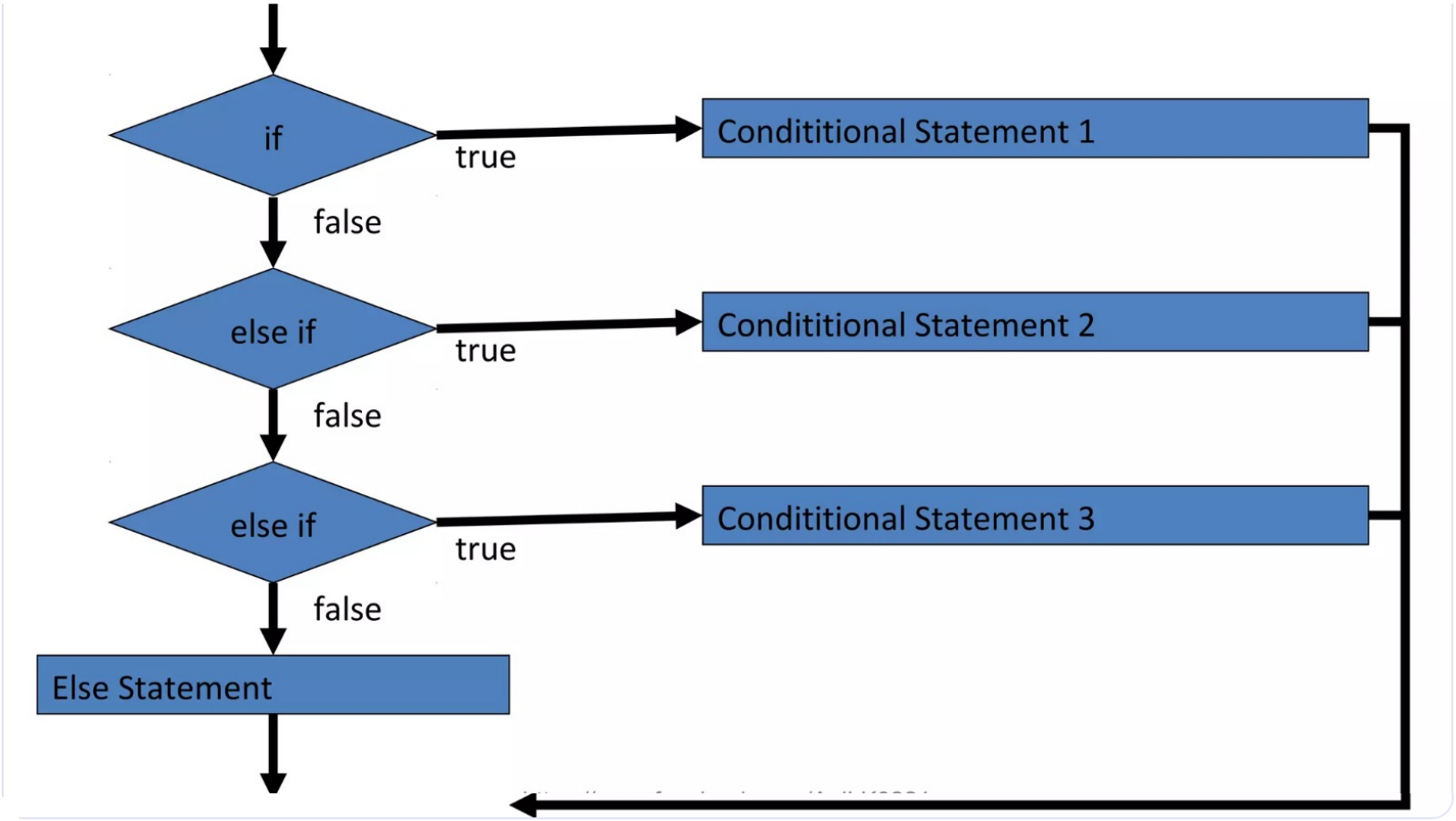
- When if statement occurs with in another if statement, then such type of if statement is called nested if statement.
-

```
if (condition1)
if (condition2)
statement-1;
else
statement-2;
else
statement-3;
```

Nested if statement

- In this program the statement if(a>c) is nested with in the if(a>b).
- if (a>b) is true only then the second if statement if(a>c) is executed.
- If the first if condition is false then program control shifts to the statement after corresponding else statement.

```
if(a>=b && a>=c)
  cout<<"a is
   biggest";
elseif(b>=a &&
   b>=c)
cout<<"b is
   biggest";
Else
cout<<"c is
   biggest";
```


if
true
Condititional Statement 1
false
else if
true
Condititional Statement 2
false
else if
true
Condititional Statement 3
false
Else Statement

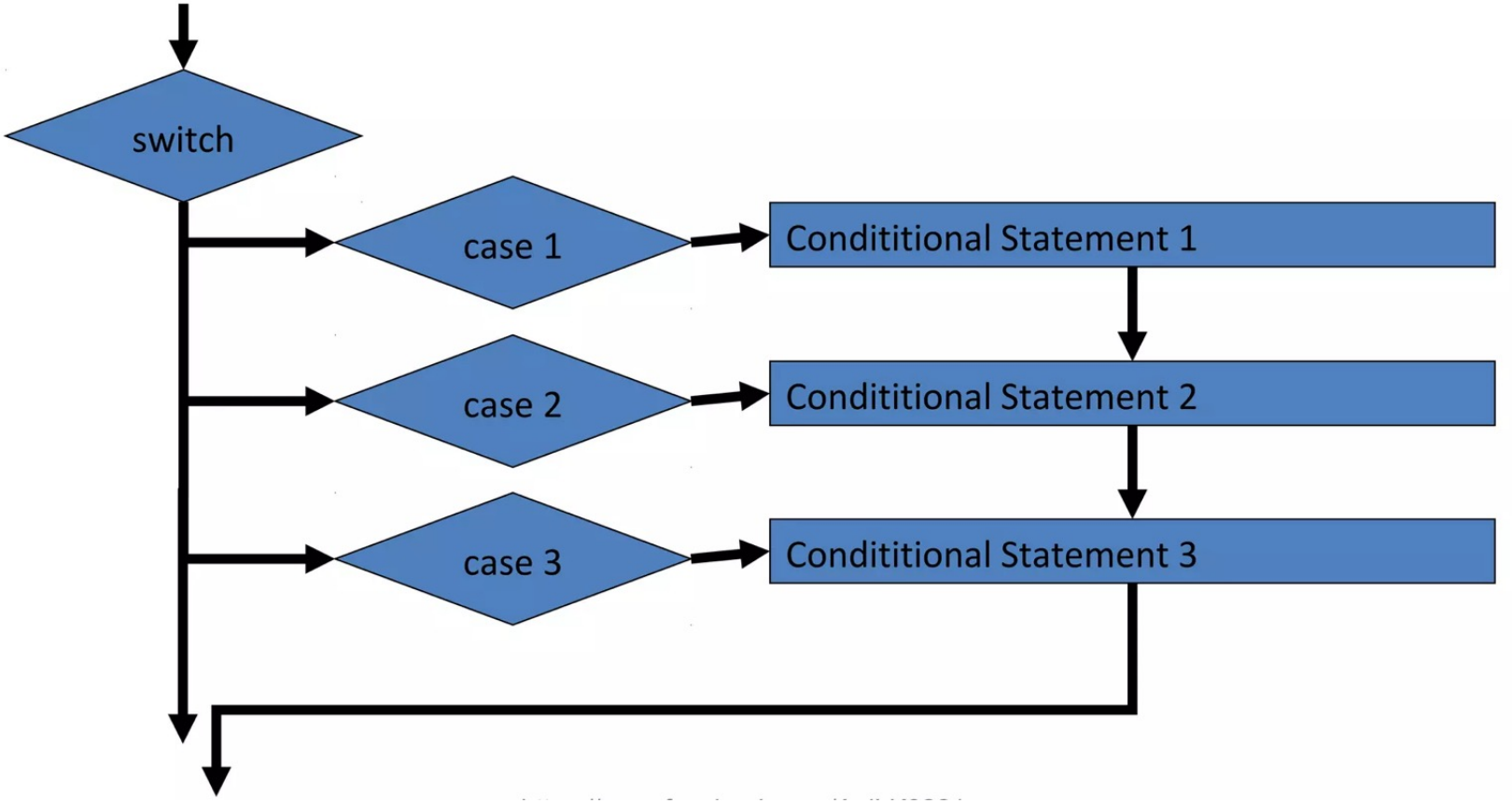
goto

- It allows making an absolute jump to another point in the program.
- "Go to" part of a program

```
#include<iostream.h>
int main()
{
int n=10;
loop:
cout<<n<<",";
n--;
if(n>0) goto loop;
cout << "FIRE!";
return 0;
}
```

Switch Statement

- Like the goto statement but more structured
  - Structure is good - less confusing
  - Its objective is to check several possible constant values for an expression and Similar to if-elseif-elseif-else but a little simpler.
  - So the switch statement is better than goto !


switch
case 1
Condititional Statement 1
case 2
Condititional Statement 2
case 3
Condititional Statement 3

Switch statement

```
switch(expression) {
case constant1:
   block of instructions 1
   break;
case constant2:
   block of instructions 2
   break;
default:
default block of instructions
}
```

Switch statement

- Switch evaluates expression and checks if it is equivalent to constant1, if it is, it executes block of instructions 1 until it finds the break keyword, then the program will jump to the end of the switch selective structure.
- If expression was not equivalent to constant1, it will check if expression is equivalent to constant2. if it is, it executes block of instructions 2 until it finds the break keyword.
- Finally if the value of expression has not matched any of the specified constants, the program will execute the instructions included in the default: section, if this one exists, since it is optional.

Switch statement

```
switch(x) {
case 1:
   cout<<"x is 1";
   break;
case 2:
   cout<<"x is 2";
   break;
   default:
 cout<<"value of x is unknown";
}
```

Loops and iterations

- Loops have as objective to repeat a certain number of times or while a condition is fulfilled. When a single statement or a group of statements will be executed again and again in the program then such type of processing is called loop. Loop is divided into two parts

- ❑ Body of loop
- ❑ Control of loop

Loops and iterations

- Control of loop is divided into two parts:
- Entry control loop- in this first of all condition is checked if it is true then body of the loop is executed. Otherwise we can exit from the loop when the condition becomes false. Entry control loop is also called base loop. Example- While loop and for loop
- Exit control loop- in this first body is executed and then condition is checked. If condition is true, then again body of loop is executed. If condition is false, then control will move out from the loop. Exit control loop is also called Derived loop. Example- Do-while

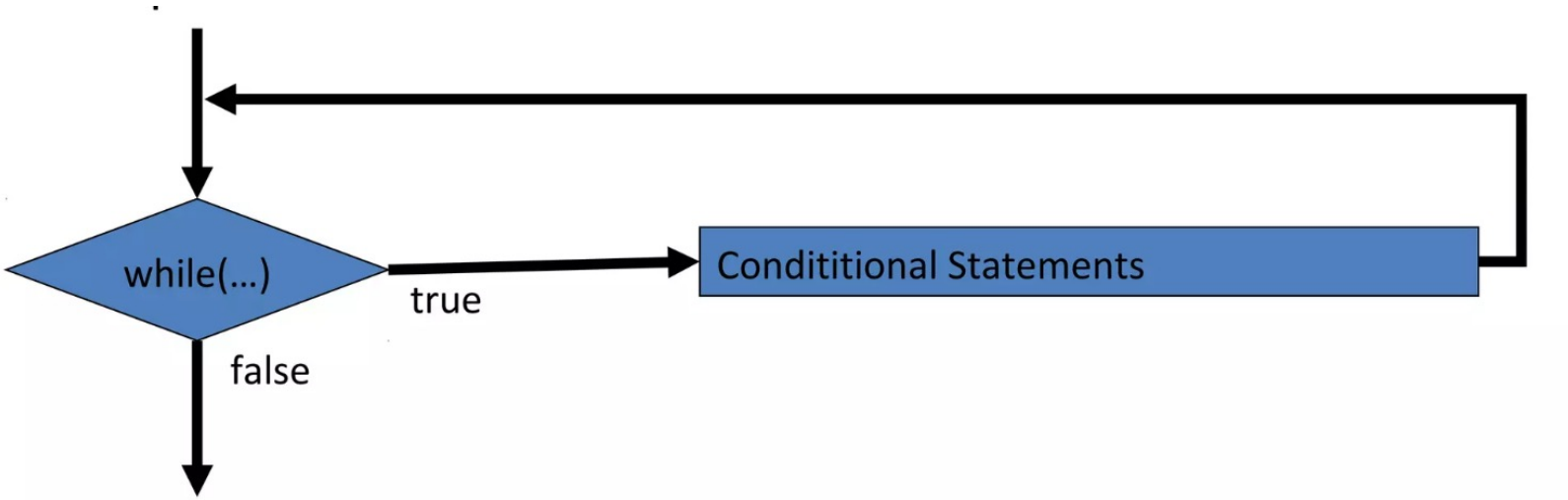
Loops and iterations

- The loops in statements in C++ language are-

- ❑While loop
- ❑Do loop/Do-while loop
- ❑For loop
- ❑Nested for loop

While Loop

- while (expression) statement
- While loop is an Entry control loop
- And its function is simply to repeat statement while expression is true.



While loop

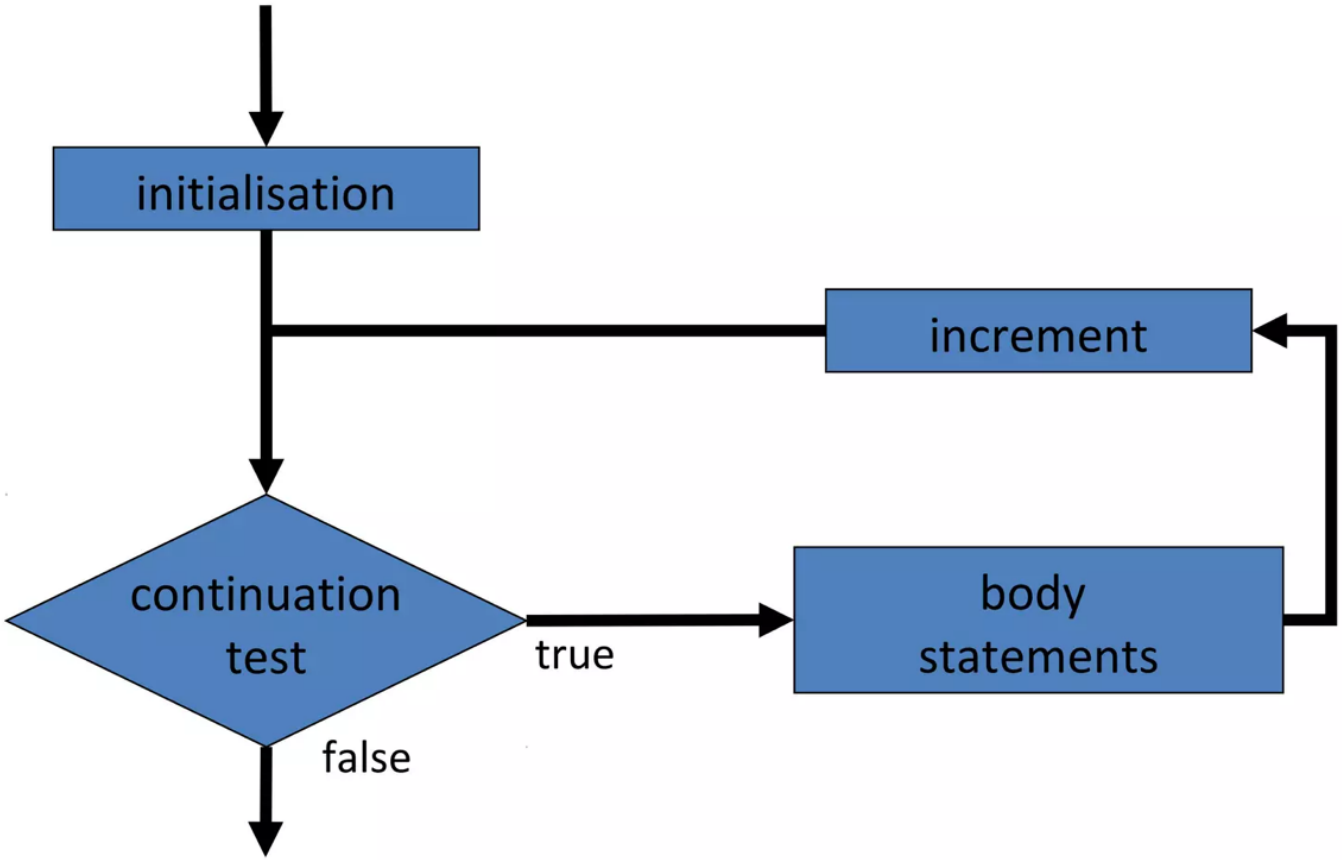
```
#include<iostream.h>
int main()
{
int n;
cout<<"Enter the starting number";
cin>>n;
while(n>0){
cout<<n<<",";
n--;
}
cout << "FIRE!";
return 0;
}
```

For loop

- For loop is an Entry control loop when action is to be repeated for a predetermined number of times.
- Most used – most complicated
- Normally used for counting
- Four parts
  - Initialise expression
  - Test expression
  - Body
  - Increment expression

```
for(initialization;condition;increment)
```

for loop

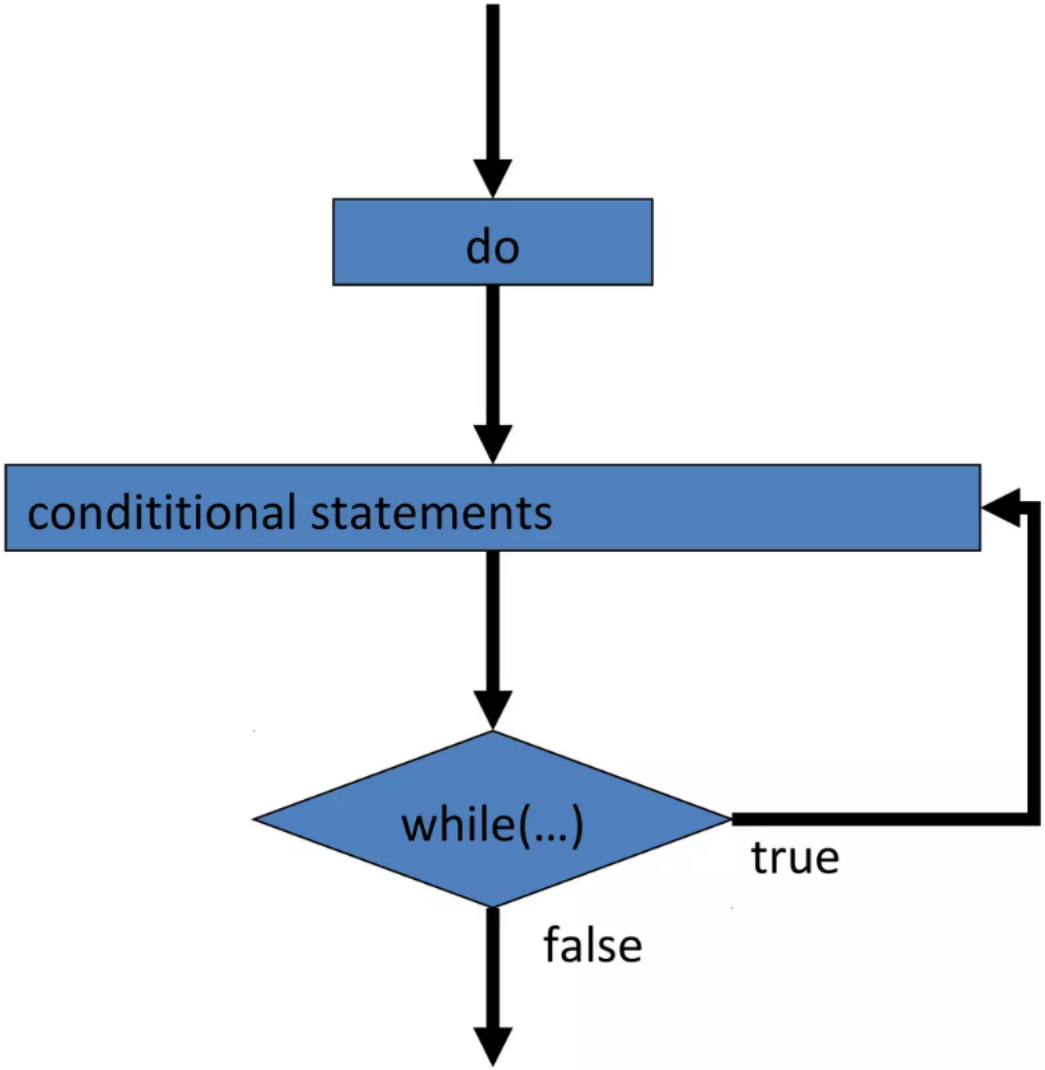

initialisation
increment
continuation test
true
body statements
false

Do-While Loop

- do statement while (expression)
- Do-while is an exit control loop. Based on a condition, the control is transferred back to a particular point in the program.
  - Similar to the while loop except that condition in the do while is check is at end of loop not the start

```
do {
    action1;
}
 while (condition is true);
    action2;
```

Do Flow



Nesting of Loops

- A loop can be inside another loop. C++ can have at least 256 levels of nesting.

```
for(init;condition;increment)
{
for(init;condition;increment)
}
statement(s);
}
statement(s);
}
```

Break Statement

- Goes straight to the end of a do, while or for loop or a switch statement block,

```
#include<iostream.h>
int main(){
int n;
for(n=10;n>0;n--){
cout<<n<<",";
if(n==5){
cout<<"count down aborted!";
break;}}
return 0;}
O/P: 10,9,8,7,6,5,count down
aborted!
```

Continue Statement

- Goes straight back to the start of a do, while or for loop,

```
#include<iostream.h>
int main()
{
int n;
for(n=10;n>0;n--){
if(n==5)continue;
cout<<n<<",";}
cout << "FIRE!";
return 0;}
O/P: 10,9,8,7,6,4,3,2,1,FIRE!
```

Summary

- Programs: step-by-step instructions that tell a computer how to perform a task
- Programmers use programming languages to communicate with the computer
  - First programming languages were machine languages
  - High-level languages can be used to create procedure-oriented programs or object-oriented programs
- Algorithm: step-by-step instructions that accomplish a task (not written in a programming language)
  - Algorithms contain one or more of the following control structures: sequence, selection, and repetition

- Sequence structure: process the instructions, one after another, in the order listed
- Repetition structure: repeat one or more instructions until some condition is met
- Selection structure: directs the computer to make a decision, and then to select an appropriate action based on that decision